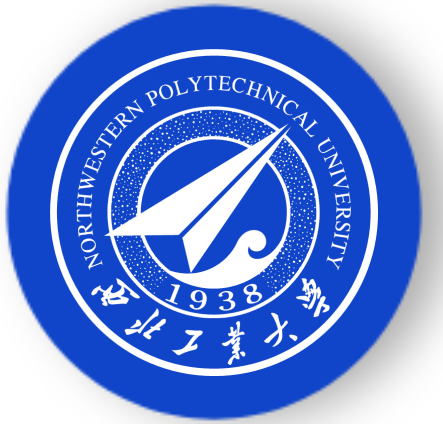


第六章 贪心算法

Chapter 6 Greedy Algorithm

计算机学院 潘永生

- 找硬币

  (0.25, 0.1, 0.05, 0.01)找0.63

  (0.11, 0.05, 0.01）找0.15

- 贪心算法总是做出在当前看来最好的选择。
- 贪心算法并不是从整体最优考虑，它做出的选择只是在某种意义上的局部最优解。
- 虽然贪心算法不能对所有问题都得到整体最优解，但是对许多问题它能产生整体最优解。一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。

活动安排问题就是要在所给的活动集合中选出最大的相容活动子集合，是可以用贪心算法有效求解的很好例子。该问题要求高效地安排一系列争用某一公共资源的活动。贪心算法提供了一个简单、漂亮的方法使得尽可能多的活动能兼容地使用公共资源。

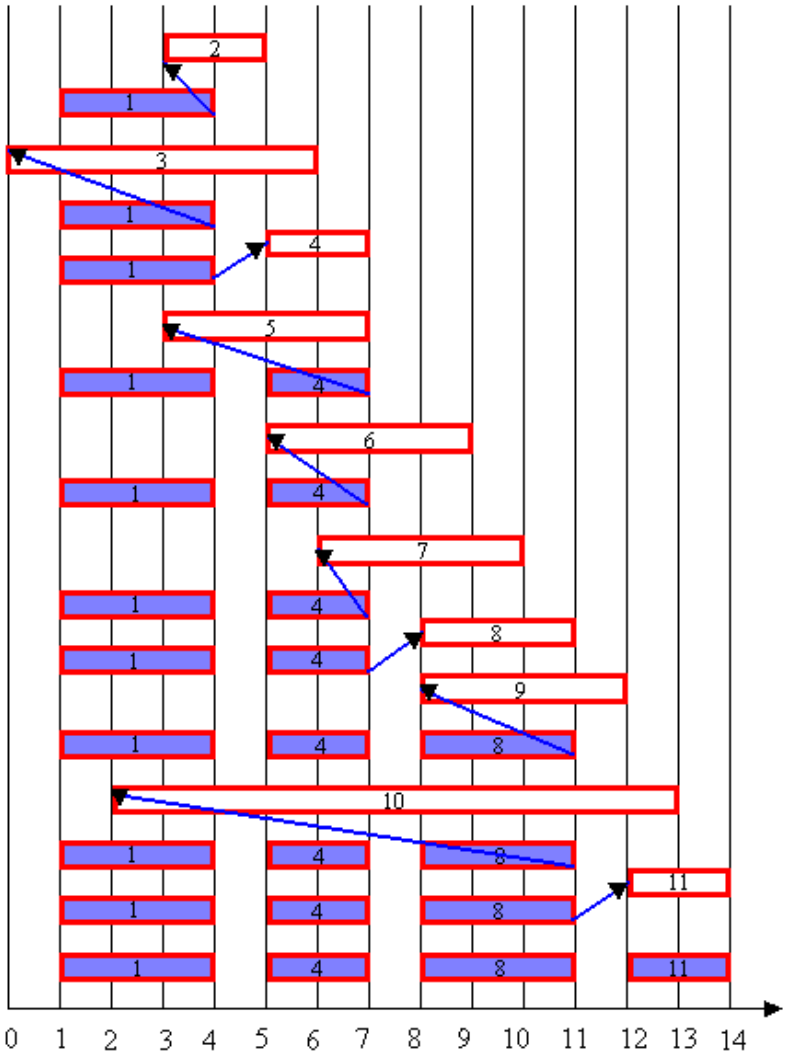
设有n个活动的集合E={1,2,…,n}，其中每个活动都要求使用同一资源，如演讲会场等，而在同一时间内只有一个活动能使用这一资源。每个活动i都有一个要求使用该资源的起始时间si和一个结束时间fi,且si <fi 。如果选择了活动i，则它在半开时间区间[si, fi)内占用资源。若区间[si, fi)与区间[sj, fj)不相交，则称活动i与活动j是相容的。也就是说，当si≥fj或sj≥fi时，活动i与活动j相容。

若被检查的活动i的开始时间Si小于最近选择的活动j的结束时间fi，则不选择活动i，否则选择活动i加入集合A中。

由于输入的活动以其完成时间的**非减序**排列，所以算法greedySelector每次总是选择**具有最早完成时间**的相容活动加入集合A中。直观上，按这种方法选择相容活动为未安排活动留下尽可能多的时间。也就是说，该算法的贪心选择的意义是**使剩余的可安排时间段极大化**，以便安排尽可能多的相容活动。

例： 设待安排的11个活动的开始时间和结束时间按结束时间的非减序排列如下：

i	1	2	3	4	5	6	7	8	9	10	11
S[i]	1	3	0	5	3	5	6	8	8	2	12
f[i]	4	5	6	7	8	9	10	11	12	13	14



算法greedySelector 的计算过程如左图所示。图中每行相应于算法的一次迭代。阴影长条表示的活动是已选入集合A的活动，而空白长条表示的活动是当前正检查相容性的活动。

```
public static int greedySelector(int [] s, int [] f, boolean
    a[])
  {
    int n=s.length-1;
    a[1]=true;
    int j=1;
    int count=1;
    for (int i=2;i<=n;i++) {
      if (s[i]>=f[j]) {
        a[i]=true;
        j=i;
        count++;
        }
      else a[i]=false;
      }
    return count;
  }
```

各活动的起始时间和结束时间存储于数组s和f中且按结束时间的非减序排列

算法greedySelector的效率极高。当输入的活动已按结束时间的非减序排列，算法只需O(n)的时间安排n个活动，使最多的活动能相容地使用公共资源。如果所给出的活动未按非减序排列，可以用O(nlogn)的时间重排。

贪心算法并不总能求得问题的**整体最优解**。但对于活动安排问题，贪心算法greedySelector却总能求得的整体最优解，即它最终所确定的相容活动集合A的规模最大。这个结论可以用数学归纳法证明。

- 设E={1,2,...,n}为所给的活动集合，E中活动按照结束时间非减序排列，故活动1具有最早结束时间。

首先证明活动安排问题的最优解以贪心选择开始，即包含活动1，设A⊆E是所给问题的一组最优解，A中活动也按结束时间非减序排列，A中第一个活动为k，若k=1，则A是以贪心选择开始的；若k>1，则设B={A-{k}}U{1}；由于f1≤fk,且A中活动是相容的，故B中的活动也是相容的。又由于B中活动个数与A中活动个数相同，故B也是最优解，可见总存在以贪心选择开始的最优活动安排方案。

选择了活动1后，原问题简化为对E中所有与1相容的活动进行安排的子问题。若A是原问题最优解，A‘=A-{1}是活动安排问题E’={i∈E; si≥f1}的最优解。如果找到E’的一个解B’，包含的活动比A’多，则将活动1加入B’产生的一个解比A更优，与假设矛盾，因此每一步所做出的贪心选择都将原问题简化为一个更小的子问题。所以贪心算法最终产生原问题的最优解。

有一批集装箱要装上一艘载重量为c的轮船。其中集装箱i的重量为Wi。最优装载问题要求确定在装载体积不受限制的情况下，将尽可能多的集装箱装上轮船。

1.算法描述

最优装载问题可用贪心算法求解。采用重量最轻者先装的贪心选择策略，可产生最优装载问题的最优解。具体算法描述如下页。

```
public static float loading(float c, float [] w, int [] x)
  {int n=w.length;
    Element [] d = new Element [n];
    for (int i = 0; i < n; i++)
      d[i] = new Element(w[i],i);
    MergeSort.mergeSort(d);
    float opt=0;
    for (int i = 0; i < n; i++) x[i] = 0;
    for (int i = 0; i < n && d[i].w <= c; i++) {
      x[d[i].i] = 1;
      opt+=d[i].w;
      c -= d[i].w;
      }
    return opt;
  }
```

2.贪心选择性质

可以证明最优装载问题具有贪心选择性质。

设集装箱依照其重量从小到大排序，（x1,..xn）为该问题一个最优解，

设如果给定问题最优装载问题有解，则1≤k≤n

当k=1时，$k=\min_{1\le k\le n}\{i \mid x_i=1\}$是满足贪心选择的最优解；

当k>1时，取y1=1,yk=0,yi=xi,1<i≤n,i≠k则

$$\sum_{i=1}^{n} wiyi = w1 - wk + \sum_{i=1}^{n} wixi \le \sum_{i=1}^{n} wixi \le c$$

可见（y1,..,yn）是问题最优解，所以最优装载满足贪心选择性质

3.最优子结构性质

设（x1,…,xn）是满足贪心选择性质的最优解，则x1=1，且（x2,…,xn）是载重量c-w1装载问题的最优解。可见，最优装载问题具有最优子结构性质。

由最优装载问题的贪心选择性质和最优子结构性质，容易证明算法loading的正确性。

算法loading的主要计算量在于将集装箱依其重量从小到大排序，故算法所需的计算时间为 O(nlogn)。

本节着重讨论可以用贪心算法求解的问题的一般特征。

对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢?这个问题很难给予肯定的回答。

但是，从许多可以用贪心算法求解的问题中看到这类问题一般具有2个重要的性质：**贪心选择性质**和**最优子结构性质**。

贪心算法
- 贪心选择性质：整体问题最优解可以通过一系列局部最优选择，自上而下
- 最优子结构性质

动态规化：自下而上

1.贪心选择性质

所谓**贪心选择性质**是指所求问题的**整体最优解**可以通过一系列**局部最优**的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。

动态规划算法通常以**自底向上**的方式解各子问题，而贪心算法则通常以**自顶向下**的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

2.最优子结构性质

当一个问题的最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。

3.贪心算法与动态规划算法的差异

贪心算法和动态规划算法都要求问题具有最优子结构性质，这是2类算法的一个共同点。但是，对于具有**最优子结构**的问题应该选用贪心算法还是动态规划算法求解?是否能用动态规划算法求解的问题也能用贪心算法求解?下面研究经典的**组合优化问题**，并以此说明贪心算法与动态规划算法的主要差别。

- **0-1背包问题：**

  给定n种物品和一个背包。物品i的重量是Wi，其价值为Vi，背包的容量为C。应如何选择装入背包的物品，使得装入背包中物品的总价值最大?

在选择装入背包的物品时，对每种物品i只有2种选择，即装入背包或不装入背包。不能将物品i装入背包多次，也不能只装入部分的物品i。

- **背包问题:**

  与0-1背包问题类似，所不同的是在选择物品i装入背包时，**可以选择物品i的一部分**，而不一定要全部装入背包，1≤i≤n。

这2类问题都具有**最优子结构**性质，极为相似，但背包问题可以用贪心算法求解，而0-1背包问题却不能用贪心算法求解。

用贪心算法解背包问题的基本步骤：

首先计算每种物品单位重量的价值Vi/Wi，然后，依贪心选择策略，将尽可能多的**单位重量价值最高**的物品装入背包。若将这种物品全部装入背包后，背包内的物品总重量未超过C，则选择单位重量价值次高的物品并尽可能多地装入背包。依此策略一直地进行下去，直到背包装满为止。

具体算法可描述如下页：

```
public static float knapsack(float c,float [] w, float [] v,float [] x)
  { int n=v.length;
    Element [] d = new Element [n];
    for (int i = 0; i < n; i++) d[i] = new Element(w[i],v[i],i);
    MergeSort.mergeSort(d);
    int i;
    float opt=0;
    for (i=0;i<n;i++) x[i]=0;
    for (i=0;i<n;i++) {
      if (d[i].w>c) break;
      x[d[i].i]=1;
      opt+=d[i].v;
      c-=d[i].w;
      }
    if (i<n){
      x[d[i].i]=c/d[i].w;
      opt+=x[d[i].i]*d[i].v;
      }
    return opt;   }
```

算法knapsack的主要计算时间在于将各种物品依其单位重量的价值从大到小排序。因此，算法的计算时间上界为O（nlogn）。当然，为了证明算法的正确性，还必须证明背包问题具有贪心选择性质。

对于**0-1背包问题**，贪心选择之所以不能得到最优解是因为在这种情况下，它无法保证最终能将背包装满，部分闲置的背包空间使每公斤背包空间的价值降低了。事实上，在考虑0-1背包问题时，应比较选择该物品和不选择该物品所导致的最终方案，然后再作出最好选择。由此就导出许多互相重叠的子问题。这正是该问题可用**动态规划算法**求解的另一重要特征。

实际上也是如此，动态规划算法的确可以有效地解0-1背包问题。

0-1 背包问题：给定 n 种物品和一个容量为 C 的背包，物品 i 的重量是 wi，其价值为 vi 。

问：应该如何选择装入背包的物品，使得装入背包中的物品的总价值最大?

分析：面对每个物品，我们只有选择拿取或者不拿两种选择，不能选择装入某物品的一部分，也不能装入同一物品多次

解决办法：声明一个 大小为 m[n][c] 的二维数组，m[i][j] 表示 在面对第 i 件物品，且背包容量为 j 时所能获得的最大价值 ，那么我们可以很容易分析得出 m[i][j] 的计算方法，

（1） j < w[i] 的情况，这时候背包容量不足以放下第 i 件物品，只能选择不拿 m[i][j] = m[i-1][j]

（2） j>=w[i] 的情况，这时背包容量可以放下第 i 件物品，我们就要考虑拿这件物品是否能获取更大的价值。如果拿取，m[i][j]=m[i-1][j-w[i]] + v[i]。这里的m[i-1][j-w[i]]指的就是考虑了i-1件物品，背包容量为j-w[i]时的最大价值，也是相当于为第i件物品腾出了w[i]的空间。如果不拿，m[i][j] = m[i-1][j] , 同（1）究竟是拿还是不拿，自然是比较这两种情况那种价值最大。

由此可以得到状态转移方程:

```
if(j>=w[i])
    m[i][j]=max(m[i-1][j],m[i-1][j-w[i]]+v[i]);
else
    m[i][j]=m[i-1][j];
```

i（物品编号）	1	2	3	4
w（体积）	2	3	4	5
v（价值）	3	4	5	6

背包容量为8

i/j	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0								
2	0								
3	0								
4	0								

https://blog.csdn.net/qq_38410730

i/j	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7
3	0	0	3	4	5	7	8	9	9
4	0	0	3	4	5	7	8	9	10

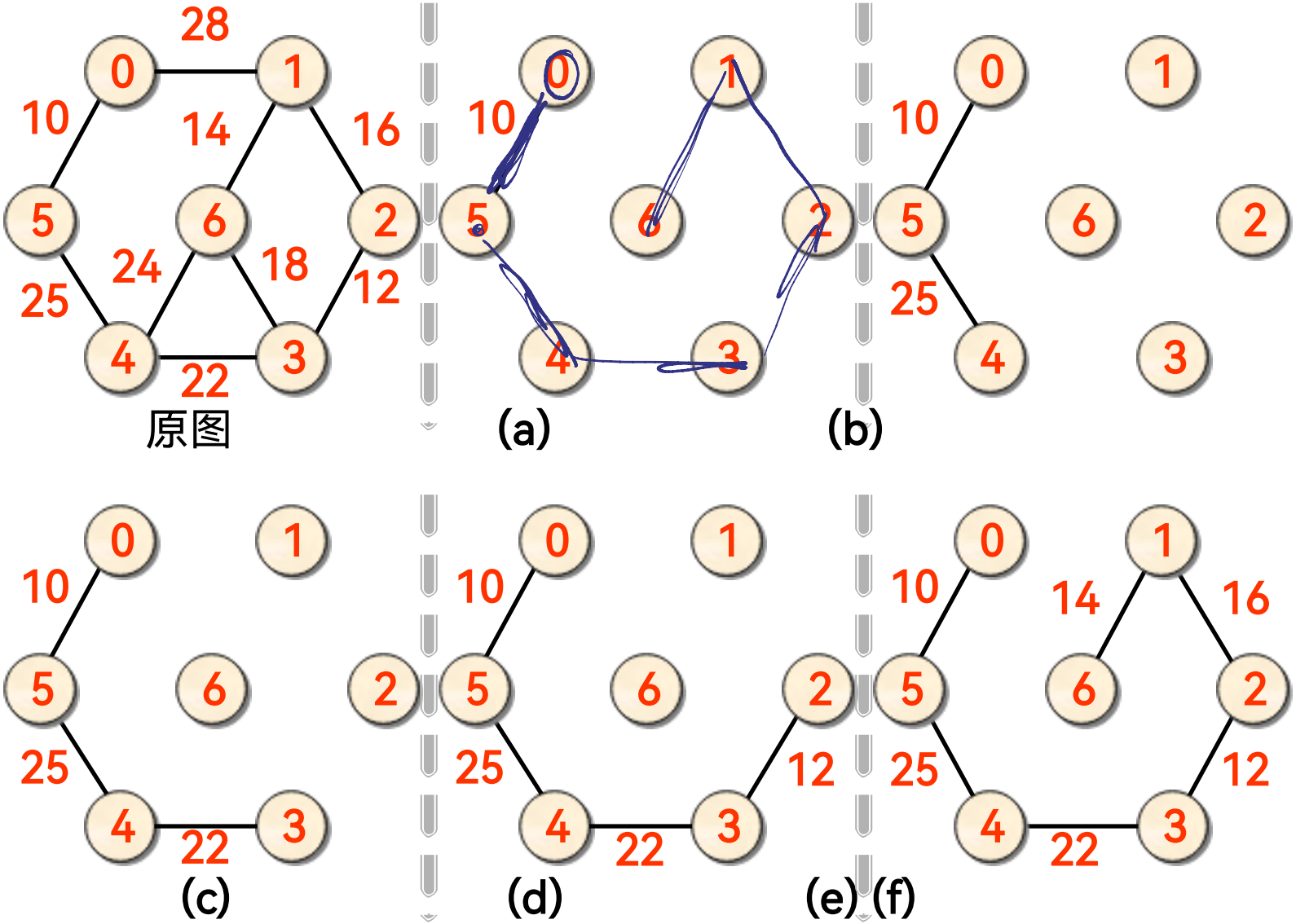
https://blog.csdn.net/qq_38410730

$j<w(i)$ $V(i,j)=V(i-1,j)$

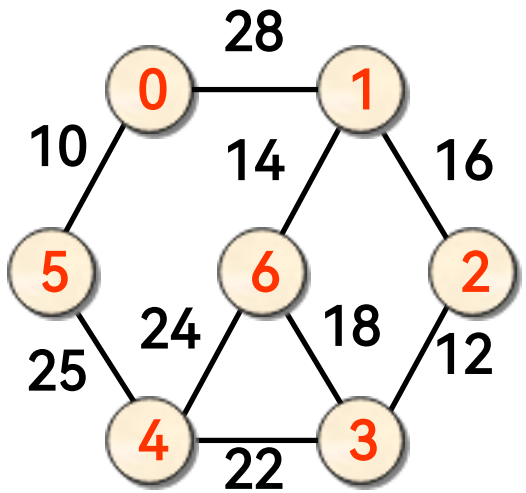
$j>=w(i)$ $V(i,j)=\max\{V(i-1,j),\ V(i-1,j-w(i))+v(i)\}$

- 使用不同的遍历图的方法，可以得到不同的生成树；从不同的顶点出发，也可能得到不同的生成树。
- 按照生成树的定义，n 个顶点的连通网络的生成树有 n 个顶点、$n-1$ 条边。
- 构造最小生成树的准则
  - 必须使用且仅使用该网络中的$n-1$ 条边来联结网络中的 n 个顶点；
  - 不能使用产生回路的边；
  - 各边上的权值的总和达到最小。

- 普里姆算法的基本思想：
  从连通网络 N 中的某一顶点 u_0 出发，选择与它关联的具有最小权值的边 (u_0, v)，将其顶点加入到生成树顶点集合U中。
  　以后每一步从一个顶点在 U 中，而另一个顶点不在 U 中的各条边中选择权值最小的边(u, v)，把它的顶点加入到集合 U 中。如此继续下去，直到网络中的所有顶点都加入到生成树顶点集合 U 中为止。
- 采用邻接矩阵作为图的存储表示。


28
0
1
10
14
16
5
6
2
24
18
12
25
4
22
3
原图
(a)
(b)
(c)
(d)
(e)
(f)

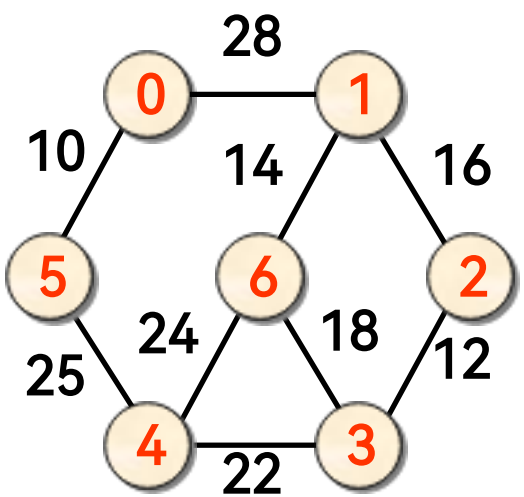
- 在构造过程中，还设置了两个辅助数组：
  - lowcost[ ] 存放生成树顶点集合内顶点到生成树外各顶点的各边上的当前最小权值；
  - nearvex[ ] 记录生成树顶点集合外各顶点距离集合内哪个顶点最近(即权值最小)。



$$\begin{pmatrix} 0 & 28 & \infty & \infty & \infty & 10 & \infty \\ 28 & 0 & 16 & \infty & \infty & \infty & 14 \\ \infty & 16 & 0 & 12 & \infty & \infty & \infty \\ \infty & \infty & 12 & 0 & 22 & \infty & 18 \\ \infty & \infty & \infty & 22 & 0 & 25 & 24 \\ 10 & \infty & \infty & \infty & 25 & 0 & \infty \\ \infty & 14 & \infty & 18 & 24 & \infty & 0 \end{pmatrix}$$

- 若选择从顶点0出发，即$u_0 = 0$，则两个辅助数组的初始状态为：

	0	1	2	3	4	5	6
lowcost	0	28	∞	∞	∞	10	∞
nearvex	-1	0	0	0	0	0	0

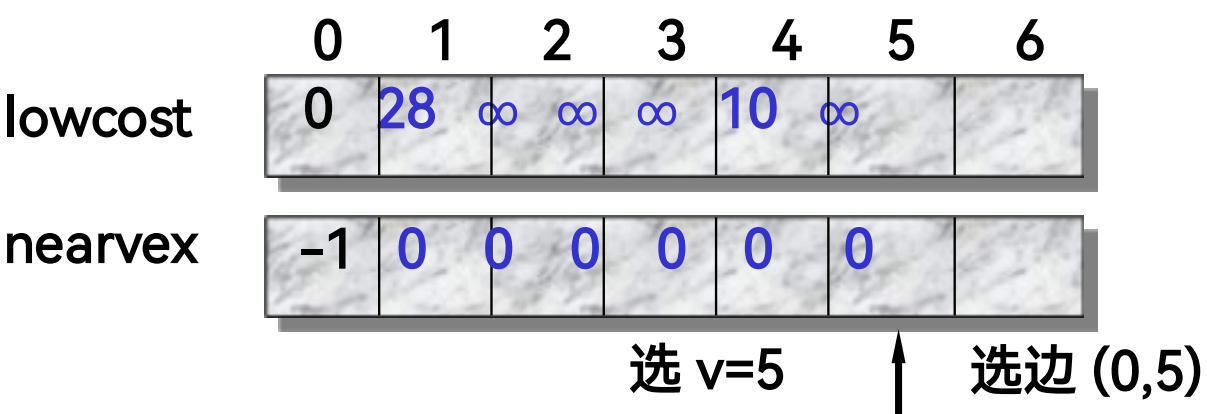


$$\begin{pmatrix} 0 & 28 & \infty & \infty & \infty & 10 & \infty \\ 28 & 0 & 16 & \infty & \infty & \infty & 14 \\ \infty & 16 & 0 & 12 & \infty & \infty & \infty \\ \infty & \infty & 12 & 0 & 22 & \infty & 18 \\ \infty & \infty & \infty & 22 & 0 & 25 & 24 \\ 10 & \infty & \infty & \infty & 25 & 0 & \infty \\ \infty & 14 & \infty & 18 & 24 & \infty & 0 \end{pmatrix}$$

然后反复做以下工作

1. 在 lowcost [ ]中选择 nearvex[i] ≠ -1 && lowcost[i]最小的边, 用 v 标记它。则选中的权值最小的边为(nearvex[v], v), 相应的权值为 lowcost[v]。
2. 将 nearvex[v] 改为-1，表示它已加入生成树顶点集合,将边 (nearvex[v], v, lowcost[v]) 加入生成树的边集合。
3. 取 lowcost[i] = min{ lowcost[i], Edge[v][i] }，即用生成树顶点集合外各顶点 *i* 到刚加入该集合的新顶点 *v* 的距离 Edge[v][i] 与原来它们到生成树顶点集合中顶点的最短距离lowcost[i] 做比较， 取距离近的作为这些集合外顶点到生成树顶点集合内顶点的最短距离。
4. 如果生成树顶点集合外顶点 i 到刚加入该集合的新顶点 v 的距离比原来它到生成树顶点集合中顶点的最短距离还要近，则修改nearvex[i] : nearvex[i] = v。表示生成树外顶点i到生成树内顶点v当前距离最近。

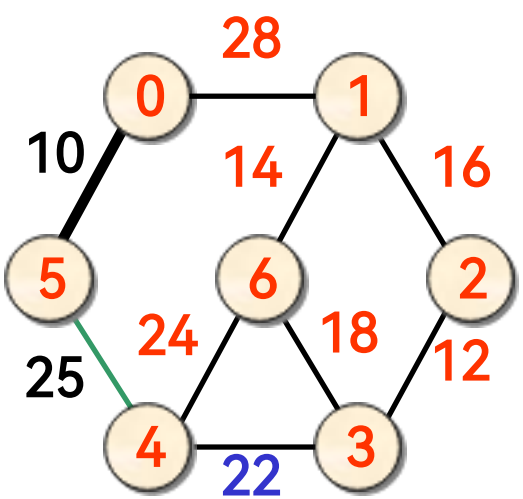
$$\begin{pmatrix} 0 & 28 & \infty & \infty & \infty & 10 & \infty \\ 28 & 0 & 16 & \infty & \infty & \infty & 14 \\ \infty & 16 & 0 & 12 & \infty & \infty & \infty \\ \infty & \infty & 12 & 0 & 22 & \infty & 18 \\ \infty & \infty & \infty & 22 & 0 & 25 & 24 \\ 10 & \infty & \infty & \infty & 25 & 0 & \infty \\ \infty & 14 & \infty & 18 & 24 & \infty & 0 \end{pmatrix}$$



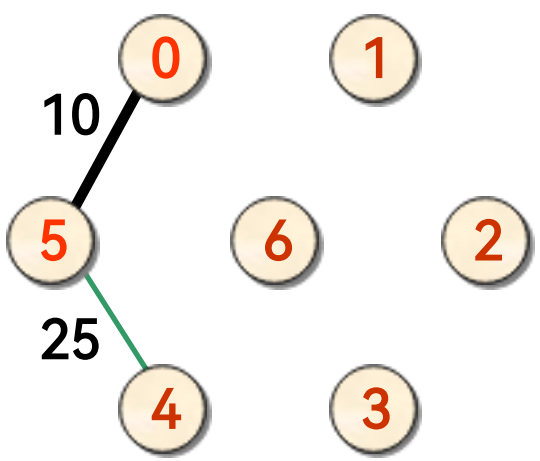
顶点v=5加入生成树顶点集合:

	0	1	2	3	4	5	6
lowcost	0	28	∞	∞	25	10	∞
nearvex	-1	0	0	0	5	-1	0

选 v=4　　选边 (5,4)

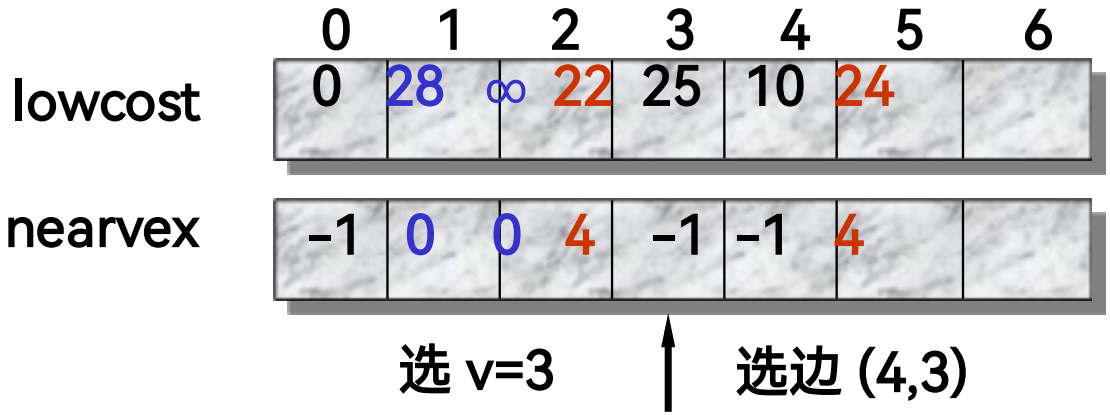


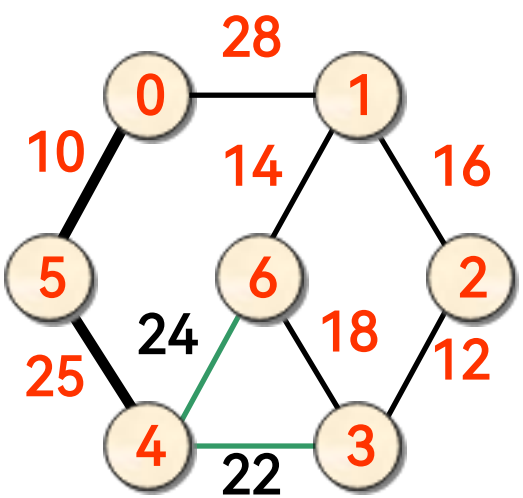
原图



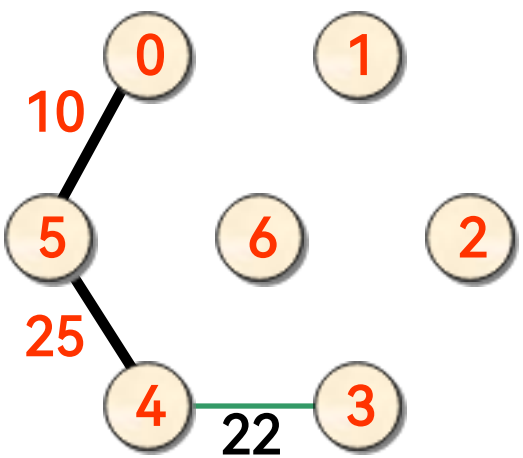
边 (0,5,10) 加入生成树

顶点v=4加入生成树顶点集合:





原图

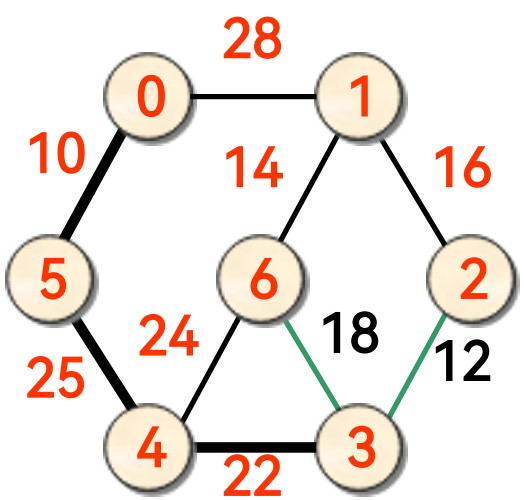


边 (5,4,25) 加入生成树

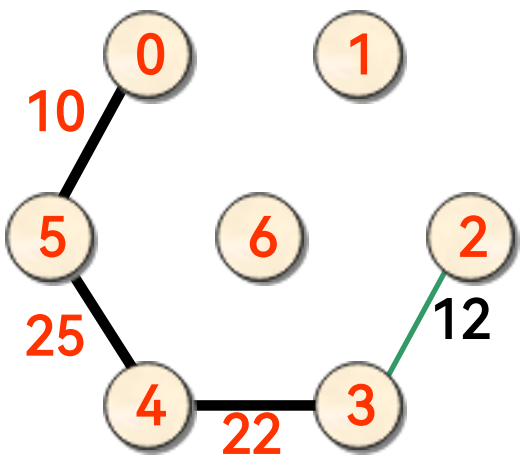
顶点v=3加入生成树顶点集合:

	0	1	2	3	4	5	6
lowcost	0	28	12	22	25	10	18
nearvex	-1	0	3	-1	-1	-1	3

选 v=2　　选边 (3,2)



原图

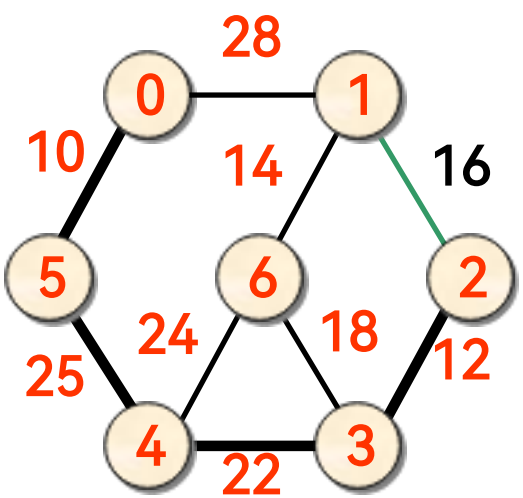


边 (4,3,22) 加入生成树

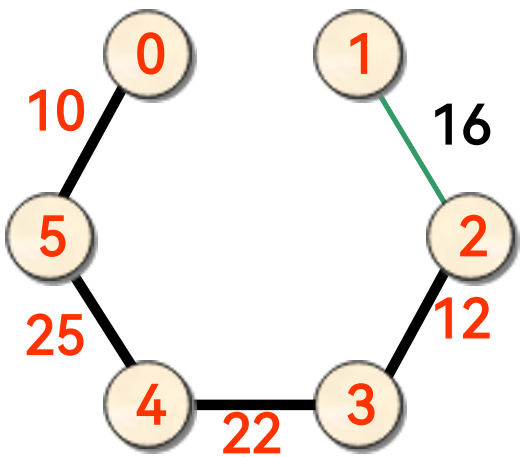
顶点v=2加入生成树顶点集合:

	0	1	2	3	4	5	6
lowcost	0	16	12	22	25	10	18
nearvex	-1	2	-1	-1	-1	-1	3

选 v=1　　选边 (2,1)



原图

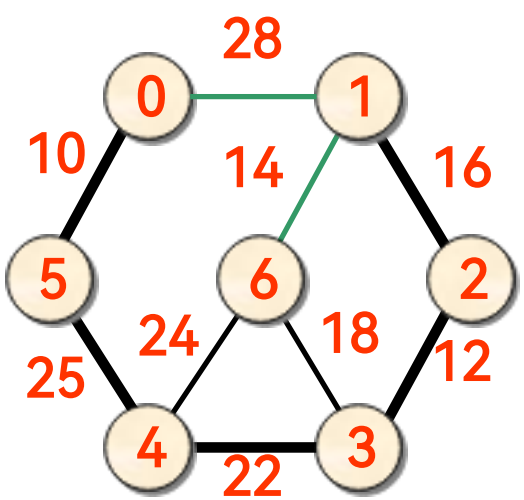


边 (3,2,12) 加入生成树

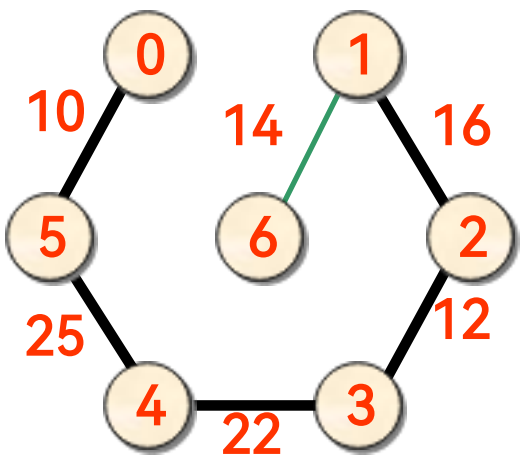
顶点v=1加入生成树顶点集合:

	0	1	2	3	4	5	6
lowcost	0	16	12	22	25	10	14
nearvex	-1	-1	-1	-1	-1	-1	1

选 v=6　　选边 (1,6)



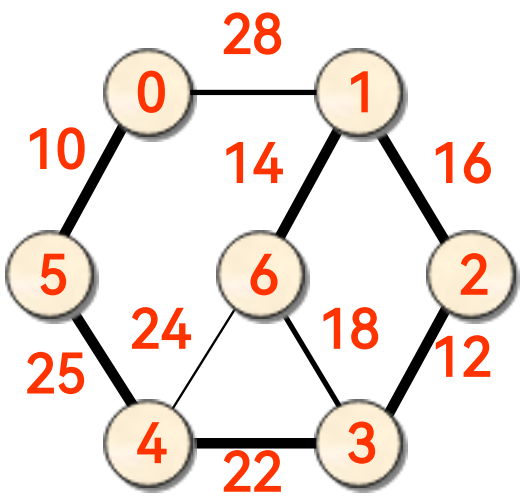
原图



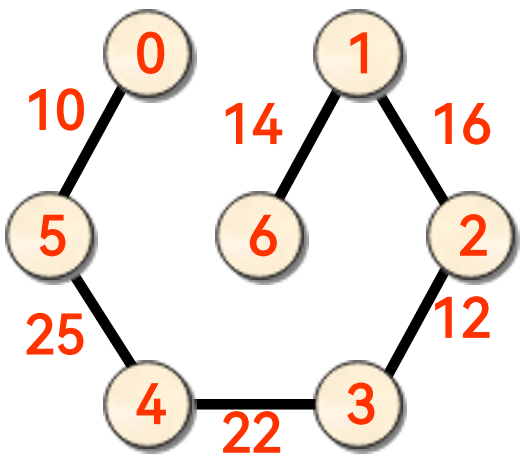
边 (2,1,16) 加入生成树

顶点v=6加入生成树顶点集合：

	0	1	2	3	4	5	6
lowcost	0	16	12	22	25	10	14
nearvex	-1	-1	-1	-1	-1	-1	-1



原图



边 (1,6,14) 加入生成树

最后生成树中边集合里存入得各条边为:

(0, 5, 10), (5, 4, 25), (4, 3, 22),
(3, 2, 12), (2, 1, 16), (1, 6, 14)

```
void Graph<string> ::
Prim ( MinSpanTree &T ) {
    int NumVertices = VerticesList.last;  //顶点数
    float * lowcost = new float[NumVertices];
    int * nearvex = new int[NumVertices];
    for ( int i = 1; i < NumVertices; i++ ) {
        lowcost[i] = Edge[0][i]; //顶点0到各边代价
        nearvex[i] = 0;              //及最短带权路径
    }
    nearvex[0] = -1;        //加到生成树顶点集合
     MSTEdgeNode e;     //最小生成树结点单元
```

```
for ( i = 1; i < NumVertices; i++ ) {
    //循环n-1次, 加入n-1条边
     float min = MAXNUM;  int v = 0;
     for ( int j = 0; j < NumVertices; j++ )
        if ( nearvex[j] != -1 && lowcost[j] < min )
       { v = j;  min = lowcost[j]; }//1
      //求生成树外顶点到生成树内顶点具有最
      //小权值的边, v是当前具最小权值的边
       if ( v ) {     //v=0表示再也找不到要求顶点
         e.tail = nearvex[v];   e.head = v;
         e.cost = lowcost[v];
         T.Insert (e); //选出的边加入生成树  2
         nearvex[v] = -1;  //该边加入生成树标记
```

```
    for ( j = 1; j < NumVertices; j++ )
        if ( nearvex[j] != -1 &&
         Edge[v][j] < lowcost[j] ) {
        lowcost[j] = Edge[v][j];   //需要修改  3
        nearvex[j] = v; //4
        }
     }//end if(v)
   }  //end for循环n-1次, 加入n-1条边
}
```

- 最短路径问题：如果从图中某一顶点(称为源点)到达另一顶点(称为终点)的路径可能不止一条，如何找到一条路径使得沿此路径上各边上的权值总和达到最小。
- 问题解法
  - 边上权值非负情形的单源最短路径问题
    — Dijkstra算法

- 问题的提法： 给定一个带权有向图*D*与源点 *v*，求从 *v* 到*D*中其它顶点的最短路径。限定各边上的权值大于或等于0。
- 为求得这些最短路径, Dijkstra提出按路径长度的递增次序， 逐步产生最短路径的算法。首先求出长度最短的一条最短路径，再参照它求出长度次短的一条最短路径，依次类推，直到从顶点v到其它各顶点的最短路径全部求出为止。

- 引入辅助数组dist。它的每一个分量dist[i]表示当前找到的从源点 v_0到终点 v_i 的最短路径的长度。初始状态：
  - 若从源点v_0到顶点 v_i有边，则dist[i]为该边上的权值；
  - 若从源点v_0到顶点 v_i 无边，则dist[i]为∞ 。
- 假设 S 是已求得的最短路径的终点的集合，则可证明：下一条最短路径必然是从v_0 出发，中间只经过 S 中的顶点便可到达的那些顶点v_x ($v_x \in V\text{-}S$)的路径中的一条。
- 每次求得一条最短路径后，其终点v_k加入集合S，然后对所有的$v_i \in V\text{-}S$，修改其 dist[i]值。

① 初始化：　$S \leftarrow \{ v_0 \}$;

dist[j] ← Edge[0][j]，　j = 1, 2, …, n–1;

// n为图中顶点个数

② 求出最短路径的长度：

dist[k] ← min { dist[i] }，　$i \in V - S$;

$S \leftarrow S \cup \{ k \}$;

③ 修改：

dist[i] ← min{ dist[i], dist[k] + Edge[k][i] },

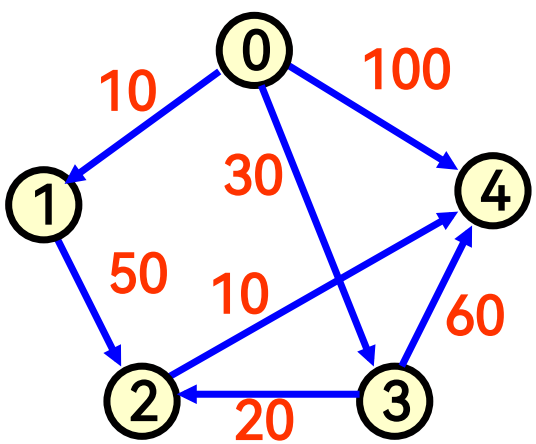
对于每一个 $i \in V - S$;

④ 判断：若 $S = V$, 则算法结束，否则转 ②。

Demo of Dijkstra algorithm

	V 1			V 2			V 3			V 4		
	$S[1]$	$d[1]$	$p[1]$	$S[2]$	$d[2]$	$p[2]$	$S[3]$	$d[3]$	$p[3]$	$S[4]$	$d[4]$	$p[4]$
0	0	10	0	0	∞	0	0	30	0	0	100	0

$$\begin{vmatrix} 0 & 10 & \infty & 30 & 100 \\ \infty & 0 & 50 & \infty & \infty \\ \infty & \infty & 0 & \infty & 10 \\ \infty & \infty & 20 & 0 & 60 \\ \infty & \infty & \infty & \infty & 0 \end{vmatrix}$$



- How to get the path from vertex 0 (source) to *i* (dest)?

  path[4] = 2 →path[2] = 3 →

  path[3] = 0,

The path is 0, 3, 2, 4, (from 0 to 4)

```
void Graph :: ShortestPath ( int n,  int v ){
//Graph是一个具有 n 个顶点的带权有向图, 各
//边上的权值由Edge[i][j]给出。本算法建立起
//一个数组: dist[j], 0≤ j<n, 是当前求到的从顶
//点 v 到顶点 j 的最短路径长度, 同时用数组
//path[j], 0≤ j<n, 存放求到的最短路径。
    for ( int i = 0; i < n; i++) {
        dist[i] = Edge[v][i];     //dist数组初始化
        S[i] = 0;
    if ( i != v && dist[i] < MAXNUM)
            path[i] = v;
        else path[i] = -1;                  //path数组初始化
    }
    S[v] = 1;   dist[v] = 0;     //顶点v加入顶点集合
```

```
//选当前不在集合S中具有最短路径的顶点u
    for ( i = 0; i < n-1; i++ ) {
        float min = MAXNUM;  int u = v;
        for ( int j = 0; j < n; j++ )
        if ( !S[j] && dist[j] < min )
          { u = j;   min = dist[j]; }
      S[u] = 1;                    //将顶点u加入集合S
      for ( int w = 0; w < n; w++ )       //修改
       if ( !S[w] && Edge[u][w] < MAXNUM
              && dist[u] + Edge[u][w] < dist[w] ) {
       //顶点w未加入S, 且通过u可以缩短路径
          dist[w] = dist[u] + Edge[u][w];
           path[w] = u;     //修改到w的最短路径
       }
    }
}
```

- 对于具有n个顶点和e条边的带权有向图，如果用带权邻接矩阵表示这个图，那么Dijkstra算法的主循环体需要O(n)时间。这个循环需要执行n-1次，所以完成循环需要$O(n^2)$时间。算法的其余部分所需要时间不超过$O(n^2)$。

多机调度问题要求给出一种作业调度方案，使所给的n个作业在尽可能短的时间内由m台机器加工处理完成。

这个问题是**NP完全问题**，到目前为止还没有有效的解法。对于这一类问题,用**贪心选择策略**有时可以设计出较好的近似算法。

约定，每个作业均可在任何一台机器上加工处理，但未完工前不允许中断处理。作业不能拆分成更小的子作业。

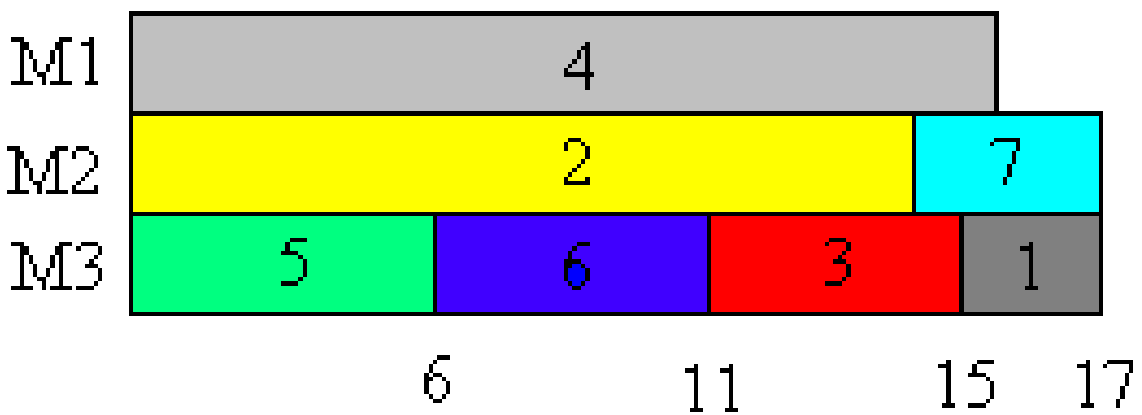
采用最长处理时间作业优先的贪心选择策略可以设计出解多机调度问题的较好的近似算法。

按此策略，当$n > m$时，只要将机器i的[0, ti]时间区间分配给作业i即可，算法只需要O(1)时间。

当$n \leq m$时，首先将n个作业依其所需的处理时间从大到小排序。然后依此顺序将作业分配给空闲的处理机。算法所需的计算时间为O(nlogn)。

按次序下一个分配最早结束的机器中

例如，设7个独立作业{1,2,3,4,5,6,7}由3台机器M1，M2和M3加工处理。各作业所需的处理时间分别为{2,14,4,16,6,5,3}。按算法greedy产生的作业调度如下图所示，所需的加工时间为17。



```
template<class Type>
void Greedy(Type a[],int n,int m)
{   if(n<=m)//机器数量比作业数量多，直接分配
    {  cout<<"直接为每个作业分配一台机器."<<endl;
       return;      }
    SelectSort(a,n);//排序，从大到小
    MinHeap<MachineNode> H(m);
    MachineNode x;
    for(int i=1; i<=m; i++)
    {  x.avail = 0;
       x.ID = i;
       H.Insert(x);   }
    for(int i=1; i<=n; i++)
    {   x = H.RemoveMin();
       cout<<"将机器"<<x.ID<<"从"<<x.avail<<"到"
          <<(x.avail+a[i].time)<<"的时间段分配给作业"
          <<a[i].ID<<endl;
       x.avail += a[i].time;
       H.Insert(x);//根据新的avail值将x插入Heap中适当位置
    }
}
```

- n>m时, 排序耗时O(nlogn). 初始化堆耗时O(m). 堆的RemoveMin和insert 共需O(nlogm).因此算法Greedy 所需时间:O(nlogn)。
- n<m时， 所需时间为O(1)。